

Transformer für Muon Track Reconstruction

Detaillierte Erklärung der Architektur und aller Scripts

Basierend auf der iceaggr-Architektur (Inar Timiryasov)

Generiert für Janik H.

März 2026

Contents

1	Überblick: Was machen wir hier eigentlich?	2
1.1	Das Problem	2
1.2	Warum ein Transformer?	2
1.3	Der Datenfluss (Big Picture)	3
2	Die Daten-Pipeline	4
2.1	Die Datenbank (SQLite)	4
2.2	MuonDataset (<code>data/dataset.py</code>)	4
2.3	Der Collator (<code>data/collators.py</code>)	5
2.3.1	Schritt für Schritt	5
3	Der Transformer (Modell-Architektur)	6
3.1	Was ist ein Transformer? (Ganz einfach)	6
3.2	Eingabe: Input Projection	6
3.3	CLS-Token: Das “Zusammenfassungs-Token”	6
3.4	RMSNorm (Root Mean Square Normalization)	6
3.5	Self-Attention mit QK-Norm	7
3.5.1	Die drei Projektionen: Q, K, V	7
3.5.2	Attention-Berechnung	7
3.5.3	Multi-Head Attention	7
3.6	Feed-Forward Network (FFN) mit ReLU ²	8
3.7	Ein Transformer-Block (Zusammen)	8
3.8	Residual Scaling + x0 Skip Connection	8
3.9	Weight-Initialisierung	9
4	Der Prediction Head (<code>models/directional_head.py</code>)	10
5	Die Loss-Funktion (<code>models/losses.py</code>)	10
5.1	Angular Distance Loss	10
6	Das Training-Script (<code>scripts/train_transformer.py</code>)	11
6.1	Überblick	11
6.2	Geometry laden	11
6.3	Optimizer: AdamW	11
6.4	Scheduler: OneCycleLR	11

6.5	Mixed Precision (AMP)	12
6.6	Early Stopping	12
6.7	Metriken	12
7	Die Embedding-Strategie im Detail	13
7.1	Warum DOM-Level statt Pulse-Level?	13
7.2	Was steckt im DOM-Vektor?	13
8	Zusammenfassung der Dateien	14
8.1	So startest du das Training	14
9	Glossar	15

1 Überblick: Was machen wir hier eigentlich?

1.1 Das Problem

In IceCube detektieren wir Neutrinos, indem wir das Cherenkov-Licht messen, das entsteht, wenn ein Neutrino mit dem Eis interagiert und dabei ein Muon erzeugt. Dieses Muon fliegt durch den Detektor und erzeugt dabei Lichtblitze an den optischen Modulen (DOMs).

Unser Ziel: Aus den gemessenen Lichtpulsen (wann hat welcher DOM wie viel Licht gesehen?) die **Flugrichtung des Muons** rekonstruieren. Die Richtung wird beschrieben durch zwei Winkel:

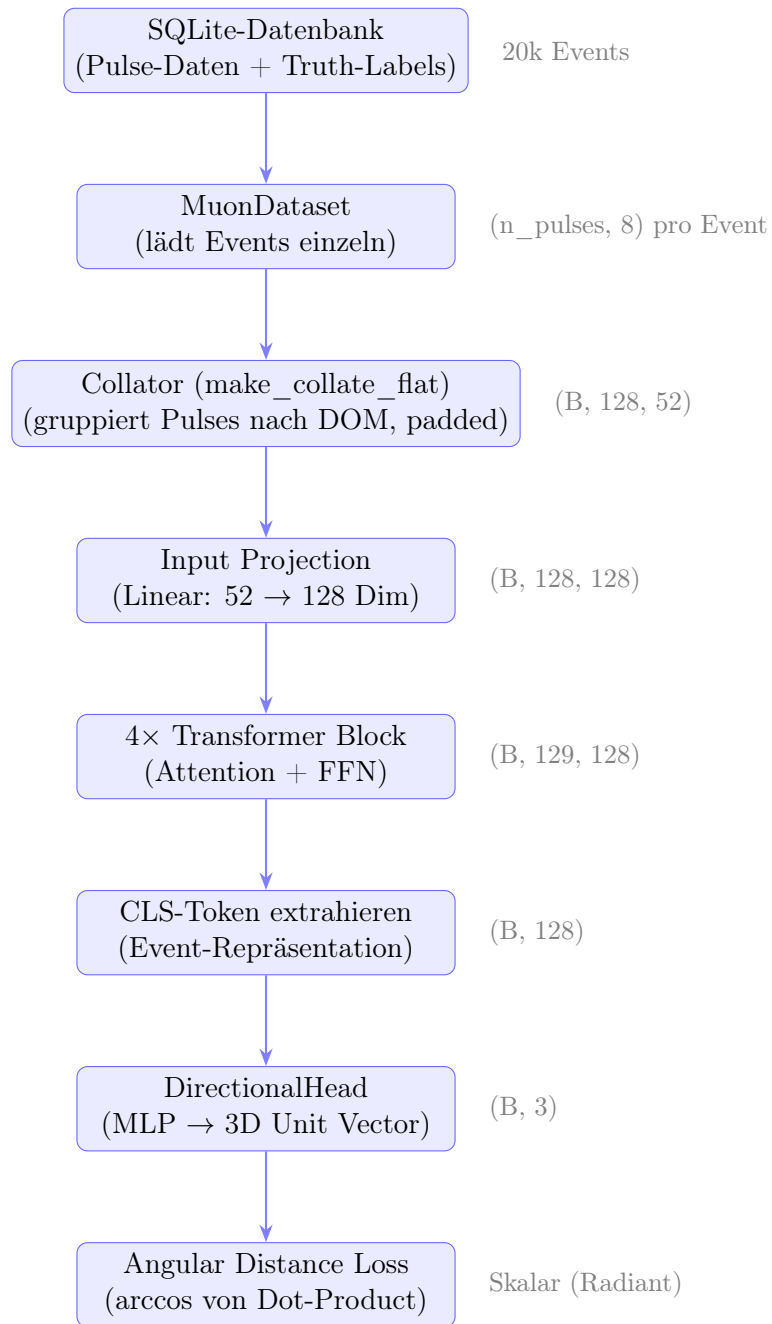
- **Zenith (θ):** Winkel zur vertikalen Achse (0° = von oben, 180° = von unten)
- **Azimuth (ϕ):** Winkel in der horizontalen Ebene (0° – 360°)

1.2 Warum ein Transformer?

Bisher haben wir Graph Neural Networks (GNNs, z.B. DynEdge) benutzt. Ein Transformer hat aber einige Vorteile:

- **Globale Aufmerksamkeit:** Jeder DOM kann direkt auf jeden anderen DOM schauen – bei GNNs muss die Information über mehrere Schichten propagieren.
- **Einfachere Architektur:** Keine Graph-Konstruktion (k-NN) nötig.
- **Skalierbarkeit:** Transformer sind gut erforscht und optimiert (Flash Attention).

1.3 Der Datenfluss (Big Picture)



2 Die Daten-Pipeline

2.1 Die Datenbank (SQLite)

Unsere Daten liegen in einer SQLite-Datenbank mit zwei Tabellen:

Tabelle	Wichtige Spalten	Bedeutung
SplitInIcePulses	dom_x, dom_y, dom_z dom_time charge hlc string, dom_number event_no	Position des DOMs (Meter) Ankunftszeit des Lichts (ns) Gemessene Ladung (PE) Hard Local Coincidence (0 oder 1) Welcher String, welcher DOM Zu welchem Event gehört der Puls
truth	zenith, azimuth energy event_no	Wahre Muon-Richtung (Radiant) Wahre Energie (GeV) Event-Nummer

Table 1: Aufbau der SQLite-Datenbank

Ein **Event** besteht aus vielen Pulsen (typischerweise 20–500), weil das Muon an vielen DOMs Licht erzeugt. Jeder Puls ist eine Zeile in `SplitInIcePulses`.

2.2 MuonDataset (data/dataset.py)

Das Dataset lädt ein Event nach dem anderen aus der Datenbank:

Listing 1: MuonDataset – vereinfacht

```
1 class MuonDataset(Dataset):
2     def __init__(self, db_path):
3         # Alle event_nos aus truth-Tabelle laden
4         self.event_nos = [...] # z.B. [1, 2, 3, ..., 20093]
5
6     def __getitem__(self, idx):
7         event_no = self.event_nos[idx]
8         # Pulse laden: SELECT dom_x, dom_y, ... WHERE event_no = ?
9         pulse_features = ... # Shape: (n_pulses, 8)
10        # Truth laden: SELECT zenith, azimuth WHERE event_no = ?
11        target = angles_to_unit_vector(zenith, azimuth) # (3,)
12        return {"pulse_features": ..., "target": ..., ...}
```

Wichtig: Jeder Worker im DataLoader öffnet seine eigene Datenbank-Connection (wegen Multi-Processing). Die DB wird im Read-Only-Modus geöffnet (`?mode=ro&immutable=1`).

Target-Konvertierung: Statt Zenith/Azimuth als Winkel zu predicten, konvertieren wir sie in einen **3D-Einheitsvektor**:

$$x = \sin(\theta) \cdot \cos(\phi) \quad (1)$$

$$y = \sin(\theta) \cdot \sin(\phi) \quad (2)$$

$$z = \cos(\theta) \quad (3)$$

Das ist besser, weil Winkel Periodizitätsprobleme haben (z.B. ist $0^\circ = 360^\circ$ für Azimuth, was der Loss-Funktion Schwierigkeiten macht).

2.3 Der Collator (data/collators.py)

Problem: Jedes Event hat eine unterschiedliche Anzahl Pulses (variable Länge). Für den Transformer brauchen wir aber einen festen Tensor pro Batch.

Lösung des Collators (angelehnt an iceaggr):

Der Collator gruppiert die Pulses **nach DOM** statt sie einzeln zu behandeln. Das ist clever, weil:

- Ein DOM typischerweise mehrere Pulses hat (Nachleuchten, Rauschen)
- Die DOM-Position nur einmal gespeichert werden muss
- Die Sequenzlänge von “Anzahl Pulses” auf “Anzahl DOMs” reduziert wird

2.3.1 Schritt für Schritt

Schritt 1: Sensor-ID berechnen: Jeder DOM wird identifiziert durch `string * 60 + dom_number`. Das ergibt eine eindeutige Sensor-ID (0–5159).

Schritt 2: Pulses nach DOM gruppieren: Alle Pulses mit der gleichen (Event, Sensor-ID)-Kombination gehören zum selben DOM im selben Event.

Schritt 3: Maximal K Pulses pro DOM: Pro DOM werden nur die ersten $K = 16$ Pulses behalten (nach Ankunftszeit sortiert). Das begrenzt die Daten pro DOM.

Schritt 4: Pulse-Features normalisieren:

- Zeit: $(t - 10000)/30000$ (typische Zeiten: 5000–40000 ns)
- Charge: $\log_{10}(\text{charge})/3$ (typisch: 0.1–1000 PE)
- HLC: $\text{hlc} - 0.5$ ($0/1 \rightarrow -0.5/+0.5$)

Schritt 5: DOM-Vektor zusammenbauen: Für jeden DOM wird ein flacher Vektor gebaut:

$$\mathbf{v}_{\text{DOM}} = [\underbrace{x, y, z}_{\text{Position}}, \underbrace{\log(n)}_{\text{Anzahl Pulses}}, \underbrace{t_1, q_1, h_1, \dots, t_K, q_K, h_K}_{\text{K Pulse-Triplets}}]$$

Dimension: $4 + 3 \times K = 4 + 3 \times 16 = 52$.

Die Position (x, y, z) kommt aus einer vorberechneten **Geometry-Tabelle** (normalisiert auf $\approx [-1, 1]$), nicht aus den Pulse-Features!

Schritt 6: Auf max_doms padden: Jedes Event wird auf genau 128 DOMs gepaddet (mit Nullen). Eine **Padding-Mask** (Bool-Tensor) markiert, welche DOMs echt sind.

Falls ein Event mehr als 128 DOMs hat, werden die mit der **spätesten Ankunftszeit** verworfen (die frühesten sind informativer).

Output des Collators:

- `dom_vectors`: Shape $(B, 128, 52)$ – die DOM-Feature-Vektoren
- `padding_mask`: Shape $(B, 128)$ – True = echte DOMs
- `targets`: Shape $(B, 3)$ – die wahren Richtungs-Unit-Vektoren

3 Der Transformer (Modell-Architektur)

3.1 Was ist ein Transformer? (Ganz einfach)

Ein Transformer ist ein Modell, das eine **Sequenz** von Vektoren verarbeitet. In unserem Fall ist die Sequenz: die N DOMs eines Events.

Die Kernidee: **Self-Attention** – jeder DOM darf “alle anderen DOMs anschauen” und entscheiden, welche Information wichtig ist.

Analogie: Stell dir eine Gruppe von Leuten vor (DOMs). Jede Person hat eine Information. In einer Besprechung (Attention) kann jeder jeden fragen und die wichtigsten Antworten zusammenfassen.

3.2 Eingabe: Input Projection

Der Collator gibt uns DOM-Vektoren der Dimension 52. Der Transformer arbeitet aber intern mit Dimension $d_{\text{model}} = 128$. Also brauchen wir eine Projektion:

$$\mathbf{h}^{(0)} = W_{\text{proj}} \cdot \mathbf{v}_{\text{DOM}} \quad W_{\text{proj}} \in \mathbb{R}^{128 \times 52}$$

Das ist einfach eine lineare Transformation (Matrix-Multiplikation) ohne Bias. Optional kann man ein MLP (2 Schichten mit GELU) benutzen.

Danach kommt **RMSNorm** (siehe unten).

3.3 CLS-Token: Das “Zusammenfassungs-Token”

Problem: Der Transformer gibt einen Vektor *pro DOM* aus. Wir brauchen aber *einen* Vektor für das ganze Event.

Lösung: Wir fügen ein spezielles Token am Anfang der Sequenz hinzu – das **CLS-Token** (Classification Token). Es ist ein lernbarer Vektor $\mathbf{c} \in \mathbb{R}^{128}$.

Die Sequenz wird also:

$$[\mathbf{c}, \mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_N]$$

Im Transformer kann das CLS-Token über Attention alle DOMs “anschauen” und ihre Information aggregieren. Am Ende nehmen wir nur das CLS-Token als Event-Repräsentation.

Analogie: Das CLS-Token ist wie ein Moderator, der bei der Besprechung zuhört und am Ende eine Zusammenfassung schreibt.

3.4 RMSNorm (Root Mean Square Normalization)

Normalisierung stabilisiert das Training. RMSNorm ist einfacher als LayerNorm (kein Mean-Shift):

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2}}$$

Das skaliert den Vektor so, dass sein RMS-Wert 1 ist. **Keine lernbaren Parameter** (anders als bei LayerNorm, das noch γ und β hat).

Warum? Weniger Parameter, schneller, und funktioniert genauso gut.

3.5 Self-Attention mit QK-Norm

Das ist das Herzstück des Transformers. Hier wird entschieden, welche DOMs aufeinander “achten”.

3.5.1 Die drei Projektionen: Q, K, V

Jeder DOM-Vektor $\mathbf{h}$ wird dreimal projiziert:

$$\mathbf{q} = W_Q \cdot \mathbf{h} \quad (\text{Query – “Was suche ich?”}) \quad (4)$$

$$\mathbf{k} = W_K \cdot \mathbf{h} \quad (\text{Key – “Was biete ich an?”}) \quad (5)$$

$$\mathbf{v} = W_V \cdot \mathbf{h} \quad (\text{Value – “Meine eigentliche Information”}) \quad (6)$$

Analogie:

- **Query:** Du gehst in eine Bibliothek und sagst: “Ich suche etwas über Physik”
- **Key:** Jedes Buch hat einen Titel (Key)
- **Value:** Der Inhalt des Buchs
- **Attention:** Die Bücher, deren Titel am besten zu deiner Suche passt, bekommst du am meisten Gewicht

3.5.2 Attention-Berechnung

1. **Dot-Product:** $\text{score}_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{d_{\text{head}}}$

Das Skalarprodukt misst, wie ähnlich Query i und Key j sind. Division durch $\sqrt{d_{\text{head}}}$ verhindert, dass die Werte zu gross werden.

2. **QK-Norm:** Vor dem Dot-Product werden Q und K mit RMSNorm normalisiert. Das macht das Training stabiler, besonders in tiefen Modellen.
3. **Masking:** Padding-DOMs (die nicht existieren) bekommen $-\infty$ als Score, damit sie nach Softmax Gewicht 0 bekommen.

4. **Softmax:** $\alpha_{ij} = \text{softmax}_j(\text{scores}_i)$

Konvertiert Scores in Wahrscheinlichkeiten (summe = 1).

5. **Gewichtete Summe:** $\mathbf{o}_i = \sum_j \alpha_{ij} \cdot \mathbf{v}_j$

Der Output für DOM i ist die gewichtete Summe aller Values.

6. **Output-Projektion:** $\text{out} = W_{\text{proj}} \cdot \mathbf{o}$

3.5.3 Multi-Head Attention

Statt eine grosse Attention zu berechnen, teilen wir die Dimension in $H = 8$ **Heads** auf. Jeder Head hat Dimension $d_{\text{head}} = 128/8 = 16$.

Warum? Verschiedene Heads können verschiedene Dinge lernen:

- Head 1: “Welche DOMs sind räumlich nah?”

- Head 2: “Welche DOMs haben ähnliche Zeiten?”
- Head 3: “Welche DOMs haben hohe Ladung?”
- etc.

3.6 Feed-Forward Network (FFN) mit ReLU²

Nach der Attention kommt ein FFN – zwei lineare Schichten mit einer Aktivierungsfunktion dazwischen:

$$\text{FFN}(\mathbf{x}) = W_2 \cdot \text{ReLU}(W_1 \cdot \mathbf{x})^2$$

ReLU² bedeutet: erst ReLU (negative Werte = 0), dann quadrieren. Das erzeugt eine **sparsere** Aktivierung als GELU (mehr Werte werden exakt 0), was bei kleinen Modellen hilft.

Dimensionen: $128 \rightarrow 256 \rightarrow 128$.

3.7 Ein Transformer-Block (Zusammen)

Ein Block besteht aus:

$$\mathbf{x} = \mathbf{x} + \text{Attention}(\text{RMSNorm}(\mathbf{x}))$$

$$\mathbf{x} = \mathbf{x} + \text{FFN}(\text{RMSNorm}(\mathbf{x}))$$

Das “+” ist die **Residual Connection** – der Input wird zum Output addiert. Das hilft beim Gradient-Flow (Gradienten können direkt durchfließen ohne durch Attention/FFN gehen zu müssen).

Pre-Norm: Die Normalisierung kommt *vor* der Attention/FFN (nicht danach). Das ist stabiler beim Training.

3.8 Residual Scaling + x0 Skip Connection

Wir haben 4 Blocks. Vor jedem Block werden die Residuals skaliert:

$$\mathbf{x} = \lambda_{\text{resid}}^{(i)} \cdot \mathbf{x} + \lambda_{\mathbf{x}_0}^{(i)} \cdot \mathbf{x}_0$$

wobei $\mathbf{x}_0$ der *initiale* Zustand nach der Input-Projektion ist.

- λ_{resid} startet bei 1.0 (normales Residual)
- $\lambda_{\mathbf{x}_0}$ startet bei 0.1 (schwacher Skip zum Anfang)
- Beide sind **lernbar** (ein Skalar pro Layer)

Warum? In tiefen Transformer kann die Information vom Anfang “vergessen” werden. Die x0-Skip-Connection stellt sicher, dass die originalen DOM-Features immer noch verfügbar sind.

3.9 Weight-Initialisierung

Die Initialisierung der Gewichte ist wichtig für stabiles Training:

- **Q, K, V, FFN-Input:** Uniform $\sim \mathcal{U}(-s, s)$ mit $s = \sqrt{3} \cdot d_{\text{model}}^{-0.5} \approx 0.153$

- **Output-Projektionen: Zero-Init!** ($W = 0$)

Das bedeutet: Am Anfang des Trainings machen Attention und FFN *gar nichts* (Output = 0), und die Residual Connection leitet den Input einfach durch. Das Modell lernt dann langsam, was es hinzufügen soll.

- **CLS-Token:** Normal $\sim \mathcal{N}(0, 0.02)$

4 Der Prediction Head (models/directional_head.py)

Der DirectionalHead nimmt das CLS-Token (Dimension 128) und sagt die Muon-Richtung als 3D-Einheitsvektor voraus:

$$\mathbf{d} = \text{normalize}\left(\underbrace{W_2 \cdot \text{GELU}(W_1 \cdot \mathbf{h}_{\text{CLS}})}_{\text{MLP mit einer Hidden-Schicht}}\right)$$

Dimensionen: $128 \rightarrow 128 \rightarrow 3 \rightarrow \text{L2-Norm}$.

L2-Normalisierung: Der Output wird auf Länge 1 normalisiert:

$$\hat{\mathbf{d}} = \frac{\mathbf{d}}{\|\mathbf{d}\|_2}$$

Das stellt sicher, dass der Output immer auf der Einheitskugel liegt (ein gültiger Richtungsvektor).

Warum GELU statt ReLU²? Der Head ist klein (nur 2 Schichten), da braucht man keine Sparsity. GELU ist “glatter” als ReLU und funktioniert besser für kleine MLPs.

5 Die Loss-Funktion (models/losses.py)

5.1 Angular Distance Loss

Wir messen den Fehler als **Winkelabstand** (Opening Angle) zwischen dem vorhergesagten und dem wahren Richtungsvektor:

$$\mathcal{L} = \frac{1}{B} \sum_{i=1}^B \arccos(\hat{\mathbf{d}}_i \cdot \mathbf{d}_i^{\text{true}})$$

Schritt für Schritt:

1. Dot-Product: $s = \hat{\mathbf{d}} \cdot \mathbf{d}^{\text{true}}$ (Skalarprodukt zweier Einheitsvektoren = Cosinus des Winkels)
2. Clipping: $s_{\text{clip}} = \text{clamp}(s, -0.9999, 0.9999)$ (arccos explodiert bei genau ± 1)
3. Winkel: $\alpha = \arccos(s_{\text{clip}})$ (in Radiant)
4. Mittelwert: $\mathcal{L} = \text{mean}(\alpha)$

Baseline: Ein zufällig ratender Predictor hat einen mittleren Opening Angle von $\sim 90^\circ$ ($\pi/2$ Radiant). Alles unter $\sim 20^\circ$ ist schon ziemlich gut.

Warum nicht MSE auf Winkel? MSE auf Azimuth hat ein Periodizitätsproblem (der Abstand zwischen 1° und 359° wäre 358° statt 2°). Die Angular Distance auf Unit-Vektoren hat dieses Problem nicht.

6 Das Training-Script (scripts/train_transformer.py)

6.1 Überblick

Listing 2: Trainings-Ablauf (Pseudo-Code)

```
1 1. Geometry laden (DOM-Positionen)
2 2. Dataset erstellen (aus SQLite-DB)
3 3. Train/Val/Test Split (80/10/10)
4 4. Collator erstellen (mit Geometry)
5 5. Modell + Optimizer + Scheduler erstellen
6 6. Training Loop:
7   - Fuer jede Epoche:
8     - Training: Forward -> Loss -> Backward -> Update
9     - Validation: Forward -> Metriken berechnen
10    - Early Stopping pruefen
11 7. Bestes Modell laden, auf Test-Set evaluieren
12 8. Ergebnisse speichern
```

6.2 Geometry laden

Die DOM-Positionen werden aus einer Parquet-Datei geladen (aus graphnet). Sie werden normalisiert:

- x, y : Division durch 600 $\rightarrow$ Bereich $\approx [-1, 1]$
- z : $(z - 750)/1250 \rightarrow$ Bereich $\approx [-1, 1]$

Das ist wichtig, weil die rohen Koordinaten in Metern sind (bis zu 2000m), was zu numerischen Problemen führen würde.

Die Geometry wird als Lookup-Tensor (5160, 3) gespeichert, indexiert über die Sensor-ID. Der Collator benutzt diesen Tensor, um die DOM-Positionen nachzuschlagen.

6.3 Optimizer: AdamW

AdamW ist Adam mit korrektem Weight Decay:

- Learning Rate: 10^{-3} (Standard)
- Weight Decay: 0.01 (leichte Regularisierung)
- $\epsilon = 10^{-8}$

6.4 Scheduler: OneCycleLR

Der Learning-Rate-Scheduler folgt dem **One-Cycle-Schema**:

1. **Warmup** (erste 10% der Steps): LR steigt von 0 auf `max_lr`
2. **Cosine Decay** (restliche 90%): LR fällt von `max_lr` auf ~ 0

Warum? Der Warmup verhindert, dass das Modell am Anfang (wenn die Gewichte noch zufällig sind) mit einer zu hohen LR “explodiert”. Der Cosine Decay sorgt für feines Tuning am Ende.

6.5 Mixed Precision (AMP)

Automatic Mixed Precision nutzt `float16` für die Vorwärtsberechnung (schneller, weniger Speicher) und `float32` für die Gradienten-Updates (numerisch stabil).

Der `GradScaler` skaliert die Loss-Werte hoch, damit die `float16`-Gradienten nicht zu klein werden (Underflow).

6.6 Early Stopping

Wenn der Validation-Loss sich für `patience` Epochen (Standard: 5) nicht verbessert, wird das Training abgebrochen. Das beste Modell wird gespeichert.

6.7 Metriken

Die wichtigste Metrik ist der **Opening Angle** – der Winkelabstand zwischen predicted und wahrer Richtung. Wir berichten:

- **Mean:** Mittlerer Opening Angle
- **Median:** Robuster als Mean (weniger von Ausreißern beeinflusst)
- **68%-Quantil:** “1-Sigma-Auflösung” (68% der Events sind besser)

7 Die Embedding-Strategie im Detail

7.1 Warum DOM-Level statt Pulse-Level?

	Pulse-Level	DOM-Level (iceaggr)
Sequenzlänge	20–500+ Pulses	20–128 DOMs
Position	In jedem Token	Einmal pro DOM
Pulse-Info	1 Pulse pro Token	K Pulses pro Token
Attention-Kosten	$\mathcal{O}(n_{\text{pulses}}^2)$	$\mathcal{O}(n_{\text{doms}}^2)$

Table 2: Vergleich Pulse-Level vs. DOM-Level Tokenisierung

Der DOM-Level-Ansatz ist **effizienter** (kürzere Sequenz) und **physikalisch sinnvoller** (ein DOM ist eine natürliche Einheit).

7.2 Was steckt im DOM-Vektor?

$$\mathbf{v} = [\underbrace{x, y, z}_{3 \text{ Werte}}, \underbrace{\log(n)}_{1 \text{ Wert}}, \underbrace{t_1, q_1, h_1, \dots, t_{16}, q_{16}, h_{16}}_{48 \text{ Werte}}] \in \mathbb{R}^{52}$$

- (x, y, z) : Normalisierte DOM-Position aus der Geometry-Tabelle
- $\log(n)$: Logarithmus der Anzahl Pulses an diesem DOM (normalisiert)
- (t_k, q_k, h_k) : Zeit, Ladung und HLC-Flag des k -ten Pulses

Falls ein DOM weniger als 16 Pulses hat, sind die restlichen Slots mit 0 gefüllt. Falls er mehr hat, werden nur die ersten 16 behalten.

8 Zusammenfassung der Dateien

Datei	Aufgabe
data/dataset.py	Events aus SQLite laden
data/collators.py	Pulses → DOM-Vektoren, Batching, Padding
models/transformer.py	Der komplette Transformer
models/directional_head.py	CLS-Token → 3D Unit Vector
models/losses.py	Angular Distance Loss
scripts/train_transformer.py	Training, Validation, Test, Speichern

8.1 So startest du das Training

Listing 3: Training starten

```
1 cd /groups/icecube/janikh/PREP/Transformer_Muon_Track_Reco
2 python scripts/train_transformer.py
3
4 # Mit Optionen:
5 python scripts/train_transformer.py \
6     --epochs 50 \
7     --batch-size 128 \
8     --d-model 128 \
9     --num-layers 4 \
10    --lr 1e-3
```

Ergebnisse werden gespeichert in `results/transformer_run/`:

- `best_model.pt` – Das beste Modell (State Dict)
- `test_results.csv` – Predictions auf dem Test-Set
- `metrics.json` – Opening-Angle-Metriken
- `training_history.csv` – Loss pro Epoche
- `train_config.json` – Alle Hyperparameter
- `split.json` – Train/Val/Test Event-IDs

9 Glossar

Attention

Mechanismus, der für jeden Token berechnet, wie viel er von jedem anderen Token “aufnehmen” soll.

CLS-Token

Spezielles Token am Anfang der Sequenz, das als Event-Zusammenfassung dient.

DOM

Digital Optical Module – ein Lichtsensor im IceCube-Detektor.

Embedding

Repräsentation eines Inputs als Vektor fester Dimension.

FFN

Feed-Forward Network – zwei lineare Schichten mit Aktivierung.

Opening Angle

Winkelabstand zwischen zwei Richtungsvektoren.

Padding Mask

Bool-Tensor, der angibt, welche Positionen echte Daten und welche Padding (Füllung) sind.

Pre-Norm

Normalisierung *vor* Attention/FFN (stabiler als Post-Norm).

QK-Norm

RMSNorm auf Query und Key vor dem Dot-Product.

Residual Connection

Der Input wird zum Output addiert ($x + f(x)$).

RMSNorm

Normalisierung basierend auf dem Root Mean Square.

Unit Vector

Vektor der Länge 1 (liegt auf der Einheitskugel).

Zero-Init

Gewichte mit 0 initialisieren (Output-Layer).